Basics Day 7 — Session 7 (Hours 25–28)

Python Programming (Basic) • Conditionals, Loops & Loop Patterns

Session 7 Overview

- Hour 25: Conditionals — if/elif/else and boundaries
- Hour 26: while loops + sentinel patterns
- Hour 27: for loops + range()
- Hour 28: Loop patterns — counters, accumulators, min/max

Hour 25: Conditionals — if/elif/else and Boundaries

Learning Outcomes

- Write clear conditional logic
- Handle boundary values correctly
- Use if/elif/else effectively for multi-branch decisions
- Order conditions to avoid overlapping branches

Review: Basic Conditionals

The Building Blocks

```
if condition:
    # runs when condition is True
elif other_condition:
    # runs when other_condition is True
else:
    # runs when all conditions are False
```

Key Points

- Python uses **indentation** (4 spaces) to define blocks
- `elif` is short for "else if"
- `else` is optional — but often a good safety net
- Only **one** branch executes per if/elif/else chain

Condition Ordering Matters

The Problem

When conditions overlap, the **order** determines which branch runs first.

Bad Example — Overlapping Conditions

```
weight = 5

if weight <= 10:
    cost = 5.00      # This catches weight=5
elif weight <= 5:
    cost = 3.00      # Never reached for weight=5!
```

Good Example — Correct Order

```
weight = 5

if weight <= 5:
```

Nested vs Flat Conditionals

Nested (Harder to Read)

```
if age >= 18:
    if has_ticket:
        print("Welcome!")
    else:
        print("Buy a ticket first.")
else:
    print("Too young.")
```

Flat (Clearer)

```
if age < 18:
    print("Too young.")
elif not has_ticket:
    print("Buy a ticket first.")
else:
    print("Welcome!")
```

Readable Boolean Expressions

Tips for Cleaner Conditions

```
# Avoid double negatives
# Bad:
if not is_invalid:
    ...

# Better:
if is_valid:
    ...
```

Comparing to Boolean Values

```
# Don't write this:
if is_ready == True:
    ...

# Write this instead:
```

Boundary Values

What Are Boundaries?

The exact values where behavior changes — e.g., the cutoff between shipping tiers.

Common Boundary Bug

```
# Bug: What happens when weight is exactly 5?
if weight < 5:
    cost = 3.00
elif weight < 10:
    cost = 5.00
# weight=5 → cost=5.00 (is that correct?)
```

Fix: Use `<=` When the Boundary Belongs to the Lower Tier

```
if weight <= 5:
    cost = 3.00
elif weight <= 10:
```

Demo: Shipping Calculator

Watch For:

- Correct ordering of conditions
- Boundary values handled properly
- Clear output messages

```
weight = float(input("Enter package weight (kg): "))

if weight <= 0:
    print("Invalid weight.")
elif weight <= 2:
    cost = 5.00
    category = "Light"
elif weight <= 5:
    cost = 10.00
    category = "Medium"
elif weight <= 10:
    cost = 20.00
    category = "Heavy"
```

Lab: Shipping Calculator

Instructions (30 minutes)

Task: Build a shipping cost calculator with tiered pricing.

Requirements:

1. Ask the user to input a package weight (in kg)
2. Compute shipping cost using these tiers:
    - 0 kg or less → Invalid (print error message)
    - Up to 2 kg → $5.00 (Light)
    - Up to 5 kg → $10.00 (Medium)
    - Up to 10 kg → $20.00 (Heavy)
    - Over 10 kg → $35.00 (Extra Heavy)
3. Print both the category name and cost

Common Pitfalls — Hour 25

Pitfall 1: Overlapping Conditions

```
# Bug: Both conditions match weight=5
if weight <= 10:
    cost = 5.00
if weight <= 5:     # Should be elif, not if!
    cost = 3.00
```

Fix: Use `elif` to create mutually exclusive branches.

Pitfall 2: Wrong elif Order

```
# Bug: Larger range checked first catches everything
if weight <= 10:
    cost = 20.00   # weight=3 hits this!
elif weight <= 5:
    cost = 10.00   # Never reached for small weights
```

Optional Extensions — Hour 25

Extension 1: Free Shipping Threshold

Add a check: if total order is over $50, shipping is free.

```
order_total = float(input("Order total: $"))

if order_total > 50:
    print("Free shipping!")
else:
    # Apply normal tiered pricing
    ...
```

Extension 2: Fragile Surcharge

Add a boolean input: if the package is fragile, add a $5 surcharge.

```
is_fragile = input("Is the package fragile? (yes/no): ").lower() == "yes"
```

Quick Check — Hour 25

Exit Ticket Question: Why does the order of `elif` statements matter?

Model Answer: "The order matters because Python evaluates conditions top to bottom and executes the **first** branch that is True. If a broader condition appears before a narrower one, the narrower condition will never execute. Always order from most specific (smallest range) to least specific (largest range)."

Hour 26: while Loops + Sentinel Patterns

Learning Outcomes

- Use while loops for repeated prompts
- Use break and continue appropriately
- Implement sentinel value patterns (e.g., 'q' to quit)
- Avoid common infinite loop mistakes

What Is a while Loop?

Basic Structure

```
while condition:
    # body runs repeatedly while condition is True
```

Simple Example

```
count = 1
while count <= 5:
    print(count)
    count += 1

print("Done!")
```

Output

```
1
```

Sentinel Value Patterns

What Is a Sentinel?

A **sentinel value** is a special input that signals "stop looping."

Common Sentinels

- `'q'` or `'quit'` to exit a menu
- `-1` to stop entering numbers
- `''` (empty string) to finish input

Example: Input Until Quit

```
while True:
    command = input("Enter command (q to quit): ")
    if command == 'q':
        print("Goodbye!")
        break
```

break and continue

break — Exit the Loop Immediately

```
while True:
    answer = input("Type 'exit' to stop: ")
    if answer == 'exit':
        break
    print(f"You said: {answer}")

print("Loop ended.")
```

continue — Skip to the Next Iteration

```
count = 0
while count < 10:
    count += 1
    if count % 2 == 0:
        continue     # Skip even numbers
    print(count)     # Only prints odd: 1, 3, 5, 7, 9
```

Common Infinite Loop Causes

Cause 1: Forgetting to Update the Loop Variable

```
# Bug: count never changes!
count = 1
while count <= 5:
    print(count)
    # Missing: count += 1
```

Cause 2: Wrong Update Direction

```
# Bug: count starts at 10 and goes UP, so count >= 5 stays True forever
count = 10
while count >= 5:
    count += 1   # Should be count -= 1 to eventually reach < 5
```

Cause 3: Condition Always True

Demo: Password Attempts Loop

Watch For:

- Limited number of attempts
- `break` on correct password
- Lockout message after failures

```
correct_password = "python123"
max_attempts = 3
attempts = 0

while attempts < max_attempts:
    password = input("Enter password: ")
    attempts += 1

    if password == correct_password:
        print("Access granted!")
        break
else:
```

while/else Explained

How It Works

```
while condition:
    # loop body
    if something:
        break
else:
    # runs ONLY if the loop ended normally (no break)
```

Key Insight

- The `else` block runs when the `while` condition becomes False
- It does **not** run if the loop exits via `break`
- Useful for "search" patterns: break when found, else = not found

Lab: Password Prompt

Instructions (30 minutes)

Task: Build a password checker with limited attempts.

Requirements:

1. Define a correct password (e.g., `"secret123"`)
2. Ask the user for a password — up to 3 attempts
3. If correct, print "Access granted!" and stop
4. If all 3 attempts fail, print "Account locked."

Completion Criteria:

- ✅ Stops correctly on successful password entry
- ✅ Locks out after exactly 3 failed attempts

Common Pitfalls — Hour 26

Pitfall 1: Not Updating the Attempt Counter

```
# Bug: attempts never increases → infinite loop
attempts = 0
while attempts < 3:
    password = input("Enter password: ")
    if password == correct:
        print("Correct!")
        break
    # Missing: attempts += 1
```

Pitfall 2: Using = Instead of ==

```
# Bug: Assignment, not comparison!
if password = "secret":    # SyntaxError in Python

# Correct:
if password == "secret":
```

Optional Extensions — Hour 26

Extension 1: Show Remaining Attempts

```
while attempts < max_attempts:
    remaining = max_attempts - attempts
    password = input(f"Enter password ({remaining} attempts left): ")
    attempts += 1
    if password == correct_password:
        print("Access granted!")
        break
```

Extension 2: Minimum Password Length Check

```
while attempts < max_attempts:
    password = input("Enter password: ")
    if len(password) < 6:
        print("Password must be at least 6 characters.")
        continue    # Don't count this as an attempt
    attempts += 1
```

Quick Check — Hour 26

Exit Ticket Question: What variable must change in a while loop to avoid infinite loops?

Model Answer: "The **loop control variable** — the variable tested in the `while` condition — must change inside the loop body. If it never changes, the condition remains True forever and the loop runs indefinitely."

Hour 27: for Loops + range()

Learning Outcomes

- Use for loops with range()
- Explain inclusive/exclusive end in range
- Use range(n), range(a, b), and range(a, b, step)
- Choose between for and while loops appropriately

What Is a for Loop?

Basic Structure

```
for variable in sequence:
    # body runs once for each item in sequence
```

Looping Over a List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output

```
apple
banana
cherry
```

The range() Function

Three Forms of range()

`range(n)` — 0 to n-1

```
for i in range(5):
    print(i)
# Output: 0, 1, 2, 3, 4
```

`range(a, b)` — a to b-1

```
for i in range(1, 6):
    print(i)
# Output: 1, 2, 3, 4, 5
```

`range(a, b, step)` — a to b-1, stepping by step

```
for i in range(0, 10, 2):
    print(i)
```

Inclusive vs Exclusive End

The Rule

`range(start, stop)` goes up to but **does NOT include** `stop`.

Why This Matters

```
# Want numbers 1 through 10?
for i in range(1, 10):
    print(i)
# Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9
# Missing 10!

# Fix:
for i in range(1, 11):
    print(i)
# Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

> ⚠️ **The most common for-loop bug:** off-by-one errors from forgetting that

Counting with range()

Useful Patterns

Count from 1 to n:

```
n = 5
for i in range(1, n + 1):
    print(i)
# 1, 2, 3, 4, 5
```

Count backward:

```
for i in range(5, 0, -1):
    print(i)
# 5, 4, 3, 2, 1
```

Skip every other number:

```
for i in range(0, 10, 2):
```

for vs while — When to Use Each

Use for When:

- You know how many times to loop
- You're iterating over a sequence (list, range, string)
- You want cleaner, more predictable code

Use while When:

- You don't know how many iterations in advance
- You're waiting for user input or a condition to change
- You need a sentinel-based loop

Quick Comparison

```
# for: known count
for i in range(5):
```

Demo: Multiplication Table

Watch For:

- Using range(1, 11) for 1 through 10
- Formatted output alignment
- Nested loops for grid (optional, keep small)

```
n = int(input("Enter a number: "))

print(f"Multiplication table for {n}:")
print("-" * 20)

for i in range(1, 11):
    result = n * i
    print(f"{n} x {i:2d} = {result:3d}")
```

Expected Output (n = 7)

Lab: Multiplication Table

Instructions (30 minutes)

Task: Build a multiplication table generator.

Requirements:

1. Ask the user for a number `n`
2. Print the multiplication table for `n` from 1 to 10
3. Use `range()` to generate the multipliers
4. Format output so columns align

Completion Criteria:

- ✅ Correct outputs for 1 through 10
- ✅ Formatting is readable and aligned

Common Pitfalls — Hour 27

Pitfall 1: Off-by-One in range()

```
# Bug: Prints 1 to 9, not 1 to 10
for i in range(1, 10):
    print(f"{n} x {i} = {n * i}")

# Fix: Use range(1, 11) for 1 through 10
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Pitfall 2: Not Converting Input to int

```
# Bug: n is a string, can't multiply
n = input("Enter a number: ")
result = n * 3   # "777" not 21!

# Fix:
n = int(input("Enter a number: "))
```

Optional Extensions — Hour 27

Full Multiplication Grid

Print a full 1 through n grid using nested loops:

```
n = int(input("Grid size: "))

# Header row
print("    ", end="")
for j in range(1, n + 1):
    print(f"{j:4d}", end="")
print()
print("-" * (4 * n + 4))

# Table body
for i in range(1, n + 1):
    print(f"{i:3d} |", end="")
    for j in range(1, n + 1):
        print(f"{i * j:4d}", end="")
    print()
```

Quick Check — Hour 27

Exit Ticket Question: What does `range(1, 4)` produce?

Model Answer: "`range(1, 4)` produces the sequence `1, 2, 3`. The start value (1) is **inclusive** and the stop value (4) is **exclusive**. To see the values, you can use `list(range(1, 4))` which gives `[1, 2, 3]`."

Hour 28: Loop Patterns — Counters, Accumulators, Min/Max

Learning Outcomes

- Apply common loop patterns reliably
- Track min and max values through a loop
- Use counter and accumulator patterns
- Initialize variables safely before looping

Pattern 1: Counter

What Is It?

A variable that counts how many times something happens.

Template

```
count = 0
for item in collection:
    if some_condition(item):
        count += 1
print(f"Found {count} matches.")
```

Example: Count Passing Scores

```
scores = [85, 42, 91, 67, 73, 55, 88]
passing_count = 0

for score in scores:
```

Pattern 2: Accumulator

What Is It?

A variable that accumulates (adds up) values through a loop.

Template

```
total = 0
for item in collection:
    total += item
average = total / len(collection)
```

Example: Sum and Average

```
prices = [12.50, 8.75, 15.00, 6.25, 9.50]
total = 0

for price in prices:
    total += price
```

Pattern 3: Min/Max Tracking

What Is It?

Tracking the smallest and largest values seen so far.

Safe Initialization

```
# Option 1: Initialize with first item
numbers = [45, 23, 67, 12, 89, 34]
current_min = numbers[0]
current_max = numbers[0]

for num in numbers:
    if num < current_min:
        current_min = num
    if num > current_max:
        current_max = num

print(f"Min: {current_min}, Max: {current_max}")
# Min: 12, Max: 89
```

Why Not Initialize Min/Max to 0?

The Trap

```
# Bug: What if all numbers are positive?
current_min = 0    # This is smaller than any positive number!

numbers = [45, 23, 67]
for num in numbers:
    if num < current_min:
        current_min = num

print(f"Min: {current_min}")
# Output: Min: 0 ← Wrong! 0 was never in the list.
```

Safe Alternatives

```
# Option 1: Use first element
current_min = numbers[0]
```

Combining Patterns

All Together: Stats in One Loop

```
numbers = [45, 23, 67, 12, 89, 34]

count = 0
total = 0
current_min = numbers[0]
current_max = numbers[0]

for num in numbers:
    count += 1
    total += num
    if num < current_min:
        current_min = num
    if num > current_max:
        current_max = num

average = total / count
print(f"Count: {count}")
print(f"Sum: {total}")
print(f"Average: {average:.2f}")
```

Demo: Number Stats

Watch For:

- Safe min/max initialization
- Counter and accumulator in one loop
- Clean formatted output

```
numbers = []
print("Enter 5 numbers:")

for i in range(5):
    num = float(input(f"  Number {i + 1}: "))
    numbers.append(num)

# Calculate stats
total = 0
current_min = numbers[0]
current_max = numbers[0]

for num in numbers:
    total += num
    if num < current_min:
        current_min = num
```

Lab: Number Stats

Instructions (30 minutes)

Task: Build a number statistics calculator.

Requirements:

1. Ask the user for 5 numbers
2. Compute: minimum, maximum, sum, and average
3. Print all four results with clear labels

Completion Criteria:

- ✅ Correct statistics for the given input
- ✅ Clean, formatted output
- ✅ Safe initialization of min/max (not 0)

Common Pitfalls — Hour 28

Pitfall 1: Initializing Min/Max to 0

```
# Bug: 0 is not in the data
current_min = 0   # Incorrect!
current_max = 0   # Will miss negatives

# Fix: Use first element
current_min = numbers[0]
current_max = numbers[0]
```

Pitfall 2: Dividing by Wrong Count

```
# Bug: Using hardcoded count instead of actual length
average = total / 5   # What if the list has 4 items?

# Fix: Use len()
average = total / len(numbers)
```

Optional Extensions — Hour 28

Allow Variable Count with Sentinel

Let the user enter numbers until they type "done":

```
numbers = []
while True:
    entry = input("Enter a number (or 'done' to finish): ")
    if entry.lower() == 'done':
        break
    numbers.append(float(entry))

if numbers:
    total = 0
    current_min = numbers[0]
    current_max = numbers[0]

    for num in numbers:
        total += num
        if num < current_min:
            current_min = num
        if num > current_max:
            current_max = num

    average = total / len(numbers)
```

Quick Check — Hour 28

Exit Ticket Question: What's a safe way to initialize `min` before looping?

Model Answer: "Initialize `min` to the first element of the collection (`current_min = numbers[0]`), or use `float('inf')` which is larger than any real number. Never initialize to 0 because 0 might be smaller than all values in the data, giving a wrong result."

Scope Guardrail Reminder

Stay in Basics Scope

✓ if/elif/else conditional logic ✓ while loops with break/continue ✓ for loops with range() ✓ Counter, accumulator, min/max patterns ✓ Basic input/output and string formatting

Not Yet (Later in Basics)

✗ Functions — defining and calling (Session 8) ✗ Imports / modules (random, math, statistics, etc.)

Not Yet (Advanced Course)

✗ List comprehensions or generator expressions ✗ try/except error handling ✗ Lambda functions or decorators ✗ File I/O or databases

Session 7 Wrap-Up

What We Covered Today

- **Hour 25:** Conditionals — if/elif/else ordering and boundary handling
- **Hour 26:** while loops — sentinel patterns, break/continue, infinite loop avoidance
- **Hour 27:** for loops — range() with three forms, inclusive/exclusive boundaries
- **Hour 28:** Loop patterns — counters, accumulators, min/max tracking

Key Takeaways

- Order elif branches from most specific to least specific
- Always verify behavior at boundary values
- A while loop needs a changing condition to terminate
- range(start, stop) is exclusive on the upper end
- Initialize min/max safely — never to 0

Homework and Practice

Practice Tasks

1. **Shipping Calculator V2:** Add at least two more tiers and a fragile surcharge
2. **Guessing Game:** Computer picks a random number 1–100; user guesses with while loop, program says "higher" or "lower"
3. **Grade Analyzer:** Read 10 test scores, compute min, max, average, and count how many are A (90+), B (80+), etc.

Preparation for Session 8

- Review functions and how to define reusable blocks of code
- Think about how to break today's labs into smaller, reusable pieces
- Practice combining conditionals with loops

Next Session Preview: Session 8 (Hours 29–32)

Topics Coming Up

- **Hour 29:** Functions — defining, calling, returning values
- **Hour 30:** Function parameters and scope
- **Hour 31:** Functions with data structures
- **Hour 32:** Checkpoint review and practice

Get Ready To:

- Write reusable functions
- Structure programs with clear logic flow
- Combine functions with loops and data structures

Questions?

Session 7 Complete!

You've mastered:

- Writing clear, well-ordered conditional logic
- Using while loops for user-driven repetition
- Using for loops with range() for counted repetition
- Applying counter, accumulator, and min/max loop patterns

Keep practicing — see you in Session 8!